THE UNIVERSITY OF AZAD JAMMU AND KASHMIR
Department of Software Engineering

LAB REPORT 2

Activity and State Machine Diagrams

Modelling the foodPanda, Toaster and Elevator systems in UML

COURSE	Software Construction & Development (Lab)
COURSE CODE	SE-3102
SEMESTER	Fall 2026
INSTRUCTOR	Engr. Dr. Asma Javaid
TOOL USED	Visual Paradigm Community Edition
SUBMITTED BY	Sheikh Haseeb-ur-Rehman
ROLL NUMBER	2024-SE-31
DATE	October, 26 2026

Table of Contents

1. Overview 3
2. Method: How Each Diagram Was Built 3
3. Task 1: foodPanda Activity Diagram (Swimlanes) 4
3.1 Steps to Build It 4
3.2 How to Read It 4
4. Task 2: Toaster State Machine 5
4.1 Steps to Build It 5
4.2 Activities Within Each State 5
4.3 Transitions: Triggered and Triggerless 6
5. Task 3: Elevator State Machine 7
5.1 Steps to Build It 7
5.2 How to Read It 8
6. Conclusion 8

1. Overview

This lab practises two UML behavioural diagrams: the activity diagram, which shows how work flows between people or systems, and the state machine diagram, which shows how one object changes state in response to events. Three diagrams were modelled.

Table 1: Summary of lab tasks

Task	System	Diagram Type	Key Idea
Q1	foodPanda ordering	Activity diagram with swimlanes	Who does what, and in what order
Q2	Toaster	State machine	Activities inside states, triggered and triggerless events
Q3	Elevator / Lift	State machine	Movement, doors and safety states

2. Method: How Each Diagram Was Built

The same four-stage routine was followed for all three diagrams.

1. **Understand the system.** Describe it in one sentence and list who or what is involved.
2. **Identify the building blocks.** Pick out the actions (activity diagram) or the states and events (state machine).
3. **Draw in Visual Paradigm.** Create a new diagram, add the initial node, place the elements, then connect them with labelled arrows.
4. **Check and refine.** Every path must start at the initial node, every state must be reachable, and every arrow must have a clear label.

3. Task 1: foodPanda Activity Diagram (Swimlanes)

Goal: visualise the process of an online food-ordering app, showing which participant is responsible for each step.

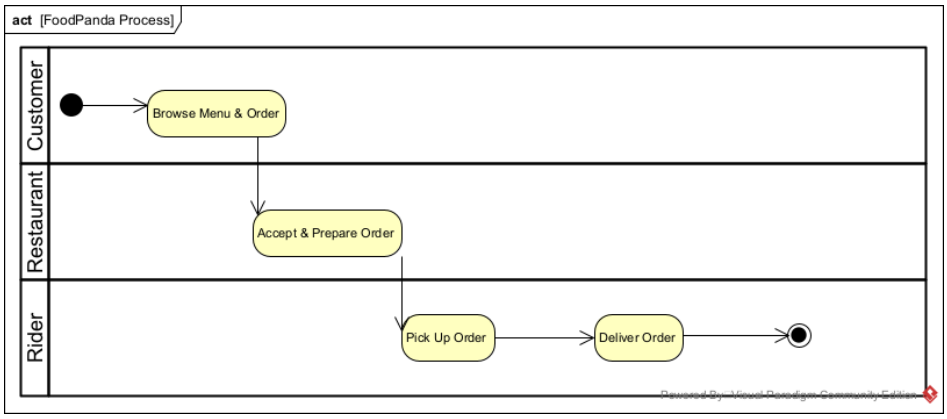


Figure 1: foodPanda activity diagram with Customer, Restaurant and Rider swimlanes

3.1 Steps to Build It

1. Create an **Activity Diagram** and name it *FoodPanda Process*.
2. Add three **swimlanes**, one per participant: *Customer*, *Restaurant* and *Rider*.
3. Place the **initial node** in the Customer lane, because the customer starts the process.
4. Add one action per step in the lane of whoever performs it.
5. Connect the actions in order with **control-flow arrows**. An arrow that crosses a lane boundary is a hand-over.
6. End the flow with a **final node** after delivery.

3.2 How to Read It

Table 2: foodPanda actions by lane

Lane	Action	What Happens
Customer	Browse Menu & Order	The customer picks items in the app and places the order
Restaurant	Accept & Prepare Order	The restaurant confirms the order and cooks it
Rider	Pick Up Order	The rider collects the food from the restaurant
Rider	Deliver Order	The rider hands the food to the customer, ending the process

The process is a simple straight line, and the swimlanes make the three hand-overs (Customer to Restaurant to Rider) easy to see at a glance.

4. Task 2: Toaster State Machine

Goal: model a toaster, list the activities inside each state, and identify triggered and triggerless events.

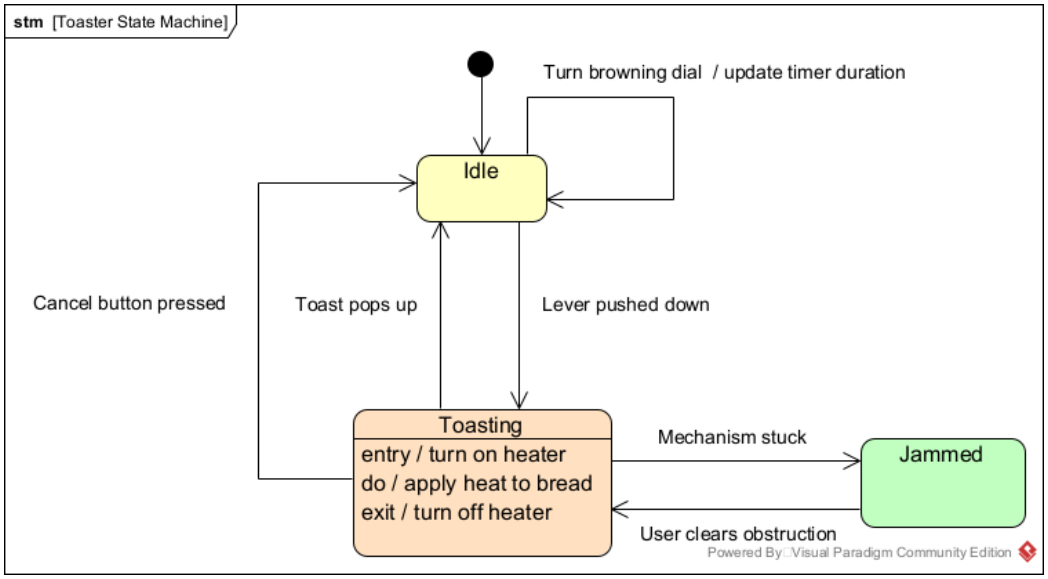


Figure 2: Toaster state machine with Idle, Toasting and Jammed states

4.1 Steps to Build It

1. Create a **State Machine Diagram** named *Toaster State Machine*.
2. Identify the states a toaster can be in: **Idle**, **Toasting** and **Jammed**.
3. Add the **initial node** and point it to *Idle*, the resting state.
4. Open the *Toasting* state and add its internal activities: **entry**, **do** and **exit**.
5. Draw a transition for each thing that can happen, and label it *event / action*.
6. Add the self-transition on *Idle* for adjusting the browning dial.

4.2 Activities Within Each State

Table 3: Entry, do and exit activities of the toaster

State	Entry	Do	Exit
Idle	None	Waits for user input	None
Toasting	Turn on heater	Apply heat to bread	Turn off heater
Jammed	None	Waits for the obstruction to be cleared	None

4.3 Transitions: Triggered and Triggerless

A triggered transition fires when an outside event occurs. A triggerless transition has no explicit event: it fires automatically when the state's own activity finishes.

Table 4: Toaster transitions

From	To	Label	Type
Idle	Idle	Turn browning dial / update timer duration	Triggered (event: dial turned, action: update timer)
Idle	Toasting	Lever pushed down	Triggered
Toasting	Idle	Toast pops up	Triggerless (fires when toasting completes)
Toasting	Idle	Cancel button pressed	Triggered
Toasting	Jammed	Mechanism stuck	Triggered
Jammed	Toasting	User clears obstruction	Triggered

Because *Toasting* has an exit action, the heater is switched off whenever the toaster leaves that state, whether the toast pops up, the user cancels, or the mechanism jams. This is the main benefit of entry and exit actions: the safety behaviour is written once and applies to every path.

5. Task 3: Elevator State Machine

Goal: model the behaviour of an elevator, including movement, door handling and the weight-limit safety check.

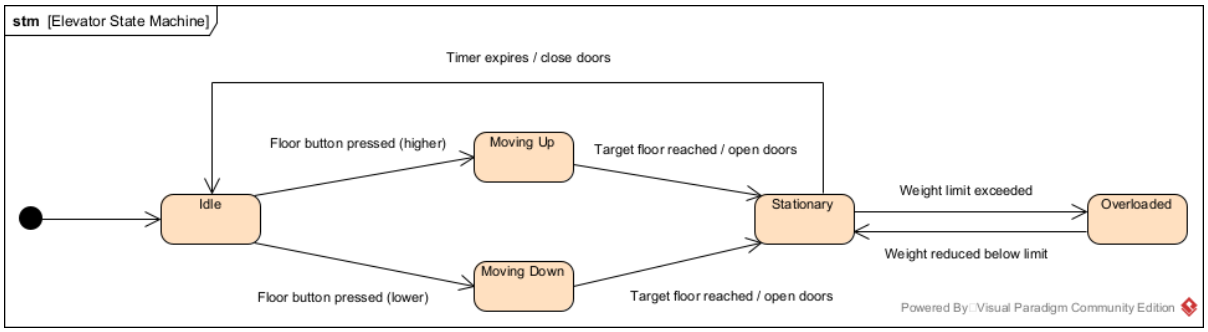


Figure 3: Elevator state machine with five states

5.1 Steps to Build It

1. Create a **State Machine Diagram** named *Elevator State Machine*.
2. List the states: **Idle**, **Moving Up**, **Moving Down**, **Stationary** (doors open at a floor) and **Overloaded**.
3. Add the **initial node** and connect it to *Idle*.
4. Add the movement transitions: a floor button pressed above or below the current floor.
5. Add the arrival transitions, which open the doors in *Stationary*.
6. Add the timer transition that closes the doors and returns the lift to *Idle*.
7. Add the safety pair of transitions between *Stationary* and *Overloaded*.

5.2 How to Read It

Table 5: Elevator states

State	Meaning
Idle	Waiting at a floor with doors closed
Moving Up	Travelling to a higher floor
Moving Down	Travelling to a lower floor
Stationary	Arrived at the target floor with doors open
Overloaded	Weight limit exceeded, so the lift cannot leave

Table 6: Elevator transitions

From	To	Event / Action
Idle	Moving Up	Floor button pressed (higher)
Idle	Moving Down	Floor button pressed (lower)
Moving Up	Stationary	Target floor reached / open doors
Moving Down	Stationary	Target floor reached / open doors
Stationary	Idle	Timer expires / close doors
Stationary	Overloaded	Weight limit exceeded
Overloaded	Stationary	Weight reduced below limit

The lift can only enter *Overloaded* from *Stationary*, so it never starts moving while overweight. It returns to *Stationary* once people step out.

6. Conclusion

The activity diagram showed *who does what* in a process, while the two state machines showed *how an object behaves over time*. Swimlanes clarified responsibility in the foodPanda flow, entry/do/exit activities and the triggered versus triggerless distinction clarified the toaster, and the elevator showed how a safety condition becomes its own state.